

AUTORZY:

- **MICHAŁ STRUGAREK (280472),**
- **JAKUB KACPRZYK (280558),**
- **SZYMON WITIW (280478)**

Raport Projektowy: Platforma Kasyno IoT na Raspberry Pi

1. Wykorzystane moduły sprzętowe

W projekcie wykorzystano zestaw laboratoryjny oparty na minikomputerze Raspberry Pi (wersja 4B). System integruje aplikację webową z fizycznymi komponentami, które pełnią rolę interfejsu użytkownika w świecie rzeczywistym.

Wykorzystane peryferia:

- **Wyświetlacz OLED (SSD1331):** Wykorzystywany do prezentowania kluczowych informacji o stanie konta gracza, kosztach gry oraz wizualizacji wyników (np. symbole slotów, wylosowany numer ruletki). Komunikacja odbywa się przez interfejs SPI.
- **Czytnik RFID (RC522):** Służy do autoryzacji i logowania użytkowników. Każda karta posiada unikalny identyfikator (UID), który jest powiązany z kontem w bazie danych.
- **Pasek LED WS2812 (8 diod RGB):** Odpowiada za zaawansowane efekty wizualne. Diody są indywidualnie adresowalne, co pozwala na tworzenie animacji takich jak „fala”, „tęcza”, „pulsowanie” czy symulacja obrotu ruletki.
- **Diody LED (4x jednobarwne):** Pełnią funkcję dodatkowej sygnalizacji świetlnej, wykorzystywanej przy powiadomieniach o wygranej/przegranej oraz jako element sekwencji startowej.
- **Buzzer:** Generuje sygnały dźwiękowe (feedback audytywny) – inne dla wygranej (sekwencja dźwięków), inne dla przegranej oraz krótkie sygnały potwierdzające logowanie.

2. Struktura oprogramowania

Projekt został zrealizowany w języku Python (backend/hardware) oraz HTML/CSS/JavaScript (frontend). Architektura opiera się na frameworku **Flask**. Poniżej przedstawiono opis poszczególnych plików i modułów:

Backend i Logika Aplikacji

- **app.py:** Główny plik uruchomieniowy serwera Flask. Odpowiada za routing (ścieżki URL), obsługę zapytań API (logowanie, start gry, pobieranie statusu) oraz zarządzanie wątkiem tła, który nasłuchuje zdarzeń z czytnika RFID.

- **games.py:** Zawiera logikę biznesową gier losowych. Zdefiniowano tu klasy SlotsGame, RouletteGame oraz MultiplierGame, które odpowiadają za matematykę losowania, obliczanie wygranych oraz koordynację animacji sprzętowych dla danej rozgrywki.
- **database.py:** Moduł obsługi bazy danych SQLite. Zawiera funkcje do tworzenia tabel (users, game_history), zarządzania saldem użytkowników oraz rejestrowania historii rozgrywek.

Obsługa Sprzętu i Integracja

- **hardware.py:** Warstwa abstrakcji sprzętowej (HAL). Plik ten izoluje logikę biznesową od bezpośrednich wywołań bibliotek niskopoziomowych (RPi.GPIO, board, neopixel). Zawiera funkcje sterujące, np. oled_show_balance(), rfid_read(), ws2812_fill().
- **animations.py:** Biblioteka efektów wizualnych. Wykorzystuje wielowątkowość (threading) do wyświetlania płynnych animacji na diodach WS2812 (np. rainbow_cycle, roulette_spin) niezależnie od głównego wątku aplikacji.
- **config.py:** Plik konfiguracyjny definiujący mapowanie pinów GPIO (BCM), ustawienia serwera oraz stałe systemowe.

Frontend (Interfejs Użytkownika)

- **templates/index.html:** Struktura strony internetowej. Zawiera panele logowania, wyboru gier oraz historii, które są dynamicznie ukrywane/pokazywane.
- **static/style.css:** Arkusze stylów zapewniające responsywność i nowoczesny wygląd (dark mode, gradienty, animacje CSS).
- **static/script.js:** Logika po stronie klienta. Obsługuje asynchroniczną komunikację z serwerem (AJAX/Fetch API), odświeża stan konta w czasie rzeczywistym i zarządza interakcją użytkownika.

3. Wkład uczestników w realizację projektu

Michał Strugarek: Backend, Logika Gier i Baza Danych (Część I - Raspberry Logic):

- Opracowanie koncepcji i zasad działania gier hazardowych (matematyka gry, mnożniki wygranych) w pliku games.py.
- Zaprojektowanie i implementacja struktury bazy danych SQLite (database.py) do przechowywania użytkowników i historii transakcji.
- Stworzenie szkieletu aplikacji we Flasku (app.py) oraz endpointów API obsługujących logikę biznesową (np. walidacja środków na koncie).
- Współautorstwo pomysłu na projekt systemu kasynowego.

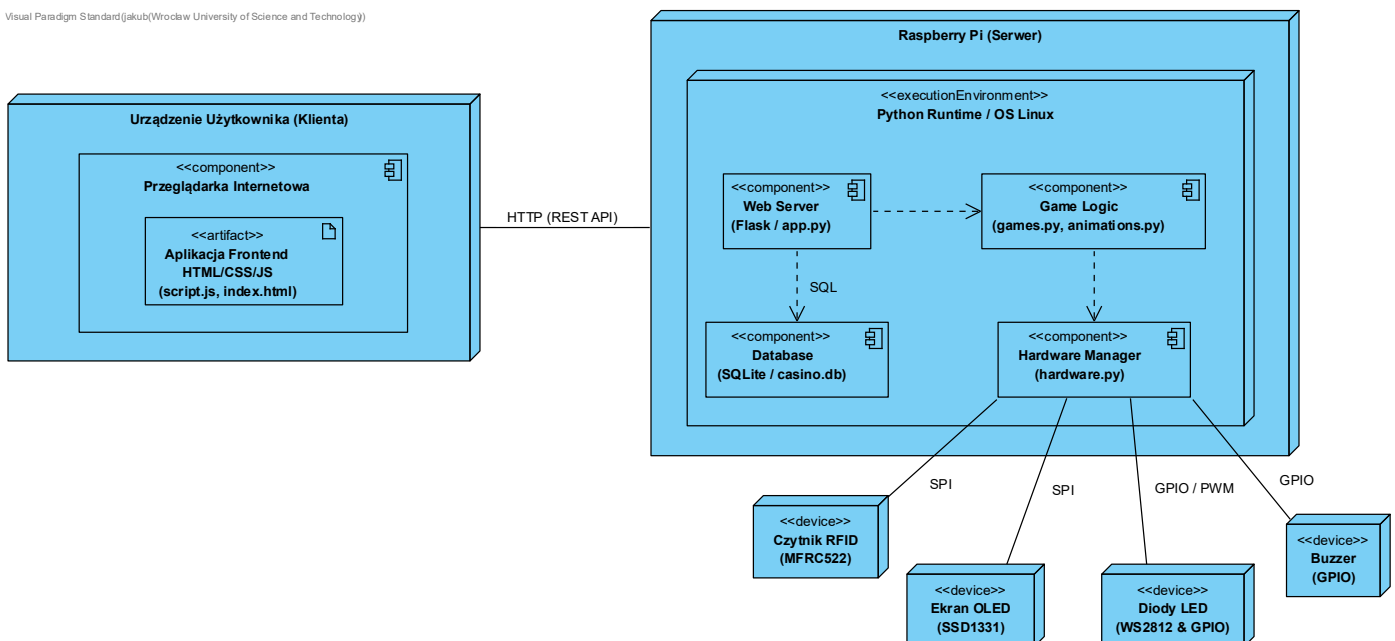
Jakub Kacprzyk: Frontend i Interfejs Webowy (Część II - Webówka):

- Zaprojektowanie i zakodowanie kompletnego interfejsu użytkownika (index.html, style.css).
- Implementacja logiki JavaScript (script.js) do dynamicznej obsługi strony bez konieczności przeładowywania (Single Page Application feel).
- Opracowanie mechanizmu cyklicznego odpytywania serwera (polling) w celu wykrycia załogowania kartą RFID.
- Stworzenie responsywnego designu i animacji interfejsu (np. spinner ładowania, efekty przycisków).
- Współautorstwo pomysłu na projekt systemu kasynowego.

Szymon Witiw: Integracja Sprzętowa i Sterowniki (Część III - Integracja z Zestawem);

- Implementacja warstwy sterowania sprzętem w hardware.py – obsługa bibliotek MFRC522 (RFID), SSD1331 (OLED) oraz RPi.GPIO.
- Programowanie zaawansowanych animacji świetlnych dla paska WS2812 w animations.py (wykorzystanie biblioteki neopixel i wątków do animacji w tle).
- Integracja sprzętu z logiką gry – stworzenie funkcji rysujących na ekranie OLED (np. oled_show_slots przy użyciu biblioteki Pillow) oraz synchronizacja dźwięku buzzera z wydarzeniami w grze.
- Konfiguracja pinów i środowiska uruchomieniowego (config.py).
- Sporządzenie niniejszego raportu końcowego oraz dokumentacji technicznej.

4. Diagram architektury



5. Od strony użytkownika

Przykładasz kartę RFID do czytnika, co powoduje piknięcie buzzera i wyświetlenie salda na małym ekranie, a strona w przeglądarce automatycznie pokazuje panel z grami.

Wybierasz grę na stronie i klikasz przycisk graj, co uruchamia kolorowe animacje świateł oraz losowanie na ekranie Raspberry Pi.

Pierwsza gra to Sloty. Jest to klasyczny jednoręki bandyta, gdzie na telefonie wciskasz przycisk Graj i czekasz na wynik losowania trzech symboli. W tym czasie diody LED na urządzeniu wykonują efekt płynącej fali, a na ekranie OLED szybko zmieniają się znaki, aż zatrzymają się w jednej linii. Jeśli wylosujesz trzy takie same symbole, na przykład trzy siódemki, terminal uruchamia tęczową animację świateł i wygrywasz dużo punktów, a przy dwóch takich samych symbolach wygrana jest mniejsza.

Druga gra to Ruletka. Tutaj interakcja jest bardziej złożona, bo przed startem musisz na telefonie wybrać z listy, czy obstawiasz kolor czerwony czy czarny, albo czy liczba będzie parzysta czy nieparzysta. Po rozpoczęciu gry na Raspberry Pi jedna dioda zaczyna biegać w kółko po pierścieniu LED, udając kulkę w ruletce, i stopniowo zwalnia. Na wyświetlaczu widać zmieniające się liczby, a gdy wirtualna kulka się zatrzyma, system sprawdza, czy obstawiony przez ciebie warunek został spełniony i wypłaca wygraną.

Trzecia gra to Mnożnik. W tej grze postawiona stawka jest mnożona przez wylosowaną liczbę, która może wynosić od zera do dziesięciu. Podczas losowania wszystkie diody na urządzeniu pulsują rytmicznie, rozjaśniając się i ściemniając, co buduje napięcie. Ekran pokazuje przelatujące wartości mnożników, a gra kończy się natychmiastowym przeliczeniem twojej stawki przez wylosowaną liczbę, więc jeśli trafisz zero, tracisz wkład, ale przy trafieniu x10 twoja wygrana jest bardzo wysoka.

Wynik finansowy od razu aktualizuje się na wyświetlaczu urządzenia oraz w przeglądarce internetowej.

Klikasz przycisk wyloguj na stronie, co zamyka dostęp do gier i sprawia, że Raspberry Pi wyświetla komunikat zachęcający do przyłożenia nowej karty.

6. Zdjęcia

